One Convolution, Three Arithmetics

Same integer multiplier; different coordinates and precision

Frank O. Kuehnel · Technical note · 30 October 2026

Abstract. Binary convolution supplies the partial products for carryless, integer/2-adic, and floating-point significand multiplication. Apparent left/right carry differences come from the ordering of place values. Reciprocal-polynomial reversal explains why an ordinary integer multiplier already computes the exact FP significand product. The decisive difference is how we treat precision: 2-adic arithmetic protects low bits; FP arithmetic rounds high significant bits using information from below. We provide ARM64 examples for all paths with integer instructions.

1. One convolution; carries follow place value

Index a normalized significand from its **leading bit toward its trailing bits**:

$$A(t) = \sum_{i=0}^{n-1} a_i t^i, \qquad B(t) = \sum_{j=0}^{n-1} b_j t^j, \qquad a_0 = b_0 = 1.$$

The bits are 0 or 1. Multiplication over the integers produces coefficient counts:

$$C(t) = A(t)B(t) = \sum_{k=0}^{2n-2} c_k t^k, \qquad c_k = \sum_{i+j=k} a_i b_j.$$

Before carrying, the same counts admit three reductions:

Arithmetic	Interpretation	Treatment of a coefficient 2
Carryless, $\mathbb{F}_2[t]$	Formal polynomial	$2t^k \mapsto 0$: XOR, no carry
Integer / 2-adic	Evaluate at $t = 2$	$2t^k \mapsto t^{k+1}$
Real significand	Evaluate at $t = 1/2$	$2t^k \mapsto t^{k-1}$

What “left” and “right” mean. Write the coefficient positions as $a_0, a_1, a_2, \ldots$, left to right. At $t = 2$, weights increase to the right, so carries go right. At $t = 1/2$, weights decrease to the right, so carries go left. In an ordinary binary word written with its most significant bit on the left, **both integer and FP significand carries go left**. In every case, two units of one place become one unit of the next greater place value.

This direction is about **place value**, not which errors are small. Higher powers of two have smaller 2-adic norm, although their ordinary place values are larger. Carrying preserves value; the norm changes how we measure precision and convergence, not the binary carry identity.

Example. For $A(t) = B(t) = 1 + t^3$, one convolution gives $C(t) = 1 + 2t^3 + t^6$. XOR yields $1 + t^6$; integer carrying yields $1 + t^4 + t^6 = 81$; real carrying yields $1 + t^2 + t^6 = 1.010001_2 = 1.265625$.

An XOR product has lost the coefficient multiplicities needed by the other two reductions. Ordinary 2-adic integers use ordinary binary carries: $\mathbb{Z}_2$ and $\mathbb{F}_2[[t]]$ are different rings. Sharing a partial-product generator with carryless arithmetic would still require its own reduction logic.

2. Reversal changes coordinates, not the multiplier

For a fixed n-bit word, define its **reciprocal polynomial**:

$$A^*(t) = t^{n-1}A(t^{-1}), \qquad M_A = A^*(2), \qquad \boxed{A(1/2) = 2^{-(n-1)}M_A.}$$

Here M_A is the unsigned integer encoded by the **original significand bits in their original MSB-at-left order**. In this coordinate change, polynomial reversal preserves the displayed bit order while changing the exponent assigned to a_i, from i to $n-1-i$. No runtime bit reversal is needed. An actual bit reversal moves the original leading bit to the rightmost position: here it would change $M_A = A^*(2)$ into $A(2)$.

Fixed-width reversal is an involution. Applying the reciprocal operation twice recovers the original polynomial:

$$\boxed{(A^*)^*(t) = t^{n-1}(t^{-1})^{n-1}A(t) = A(t).}$$

Both operations use the same width n, retaining zero coefficients even when the polynomial degree drops; the width is not recomputed from the degree.

For the four-bit significand 1.101_2 (MSB at left), let $A(t) = 1 + t + 0t^2 + t^3$, so $A^*(t) = 1 + 0t + t^2 + t^3$:

Reading	Value	Ordinary bit order
Real significand $A(1/2)$	$13/8$	1.101_2
Same polynomial at 2: $A(2)$	11	1011_2, reversed
Reciprocal polynomial at 2: $A^*(2)$	13	1101_2, original

Why multiplication works. Reverse the $2n-1$ coefficient positions of $C(t) = A(t)B(t)$, before carrying. Then

$$C^*(t) = t^{2n-2}C(t^{-1}) = A^*(t)B^*(t), \qquad \boxed{A(1/2)B(1/2) = 2^{-2(n-1)}M_AM_B.}$$

Thus ordinary integer multiplication followed by binary-point placement gives the exact real significand product. For example, $11_2 \times 11_2 = 1001_2$; restoring two fractional positions gives $10.01_2 = 2.25 = (1.1_2)^2$.

The apparent carry reversal is now explicit. Under the product-coordinate change $j = 2n-2-k$,

$$\boxed{k \longmapsto k-1 \quad \Longleftrightarrow \quad j \longmapsto j+1.}$$

The real carry toward a smaller fractional index becomes the usual integer carry toward a higher power of two. **The same integer carry machinery suffices.**

The reverse–multiply–reverse trap. Take $1.11_2 = 7/4$. Its three bits are palindromic, and integer multiplication gives $7^2 = 49 = 110001_2$. Reversing that already-carried six-bit result gives $100011_2 = 35$: after the same scaling, $35/16$, instead of $49/16$. Reversal respects multiplication of the *uncarried coefficient vectors*; carrying and bit reversal do not generally commute.

Embedding FP64 in $\mathbb{Q}_2$ is a different operation. A nonzero finite FP64 value is an exact dyadic rational $x = (-1)^s M 2^e$. The same rational embeds in $\mathbb{Q}_2$ without reversing its bits. To obtain an odd unit, factor $M = 2^v u$ and write $x = (-1)^s u 2^{e+v}$. Removing trailing zeros and adjusting the exponent preserves the value; replacing M by its bit reversal generally does not.

3. Same integer multiplication; different precision

For two normalized FP64 significands, each unsigned integer M has 53 bits:

$$m_A = M_A 2^{-52}, \qquad m_B = M_B 2^{-52}, \qquad m_A m_B = (M_A M_B) 2^{-104}.$$

The exact product needs at most **106 bits**. On ARM64, the same unsigned integer multiplication supplies both parts [1]:

```
// x0, x1: unsigned significand magnitudes
mul     x2, x0, x1       // Low 64 product bits
umulh   x3, x0, x1       // High 64 product bits; full product is x3:x2
```

The p-adic path can keep the low word and mask it. The FP64 path keeps enough of the full product to normalize and round once, with sign and exponent handled separately. **One low-word `MUL` alone does not retain a general FP64 product.** `MUL`/`UMULH` expose the required integer result without an FP multiply instruction.

Precision question	2-adic arithmetic	Floating-point arithmetic
What is preserved?	Low bits of the unit, modulo 2^r	Leading significant bits of the real magnitude
What is a small error?	Divisibility by a large power of 2	Small ordinary absolute or relative magnitude
What can be discarded?	High bits beyond the certified residue window	Low bits only after their rounding effect is known
Product boundary	Reduce modulo 2^r	Normalize, then round using discarded-bit information

A 2-adic approximation $2^v(u + O(2^r))$ certifies a congruence modulo 2^{v+r}. Unknown high bits cannot carry back into its known low bits. Reduction modulo 2^r is compatible with addition and multiplication. Once discarded, those digits cannot be recovered by widening the stored word.

An FP64 input, by contrast, denotes an exact stored dyadic value. An operation on two such inputs must round its exact real result to the chosen FP64 format. For a finite normal nearest-even result, the error is at most half an ulp; subnormals use their fixed spacing. Low product terms can carry into higher positions, and the final discarded bits decide rounding. Guard/round/sticky information must come from a correctly accumulated product, not from discarding partial products prematurely.

The difference is observable:

$$(2^{27} + 1)^2 = 2^{54} + 2^{28} + 1.$$

Modulo 2^{54}, the result is $2^{28} + 1$. FP64 nearest-even rounding gives $2^{54} + 2^{28}$. Both use the same integer product; their precision boundaries retain different information. Reversing bits or changing the exponent of the truncated residue cannot recover the discarded product in general.

Addition has the same distinction: p-adic alignment protects low valuation positions; FP alignment protects high significant positions and must account for cancellation and discarded tails. To reproduce an FP instruction sequence, round at **each instruction boundary**, rather than only at the end.

4. Inversion: shared algebra, different precision control

To invert the earlier significand $m_A = M_A 2^{-(n-1)}$, first consider $D = M_A$ odd: then $m_A^{-1} = 2^{n-1} D^{-1}$. If $M_A = 2^v D$ is even, invert its odd part D and use the scale 2^{n-1-v} instead. In either setting, refine an estimate R_k of D^{-1} using $R_{k+1} = R_k(2 - DR_k)$. The residual $e_k = 1 - DR_k$ obeys the exact identity

$$\boxed{e_{k+1} = 1 - DR_k(2 - DR_k) = (1 - DR_k)^2 = e_k^2.}$$

2-adic reciprocal. For odd D, compute $DR \equiv 1 \pmod{2^r}$. Starting with $R_0 = D$ gives three correct low bits because $D^2 \equiv 1 \pmod 8$. Five Newton–Hensel updates suffice for 64 bits [2]. Keep low bits: a small residual has many factors of two. Low-word products and wraparound preserve the required congruence.

Real reciprocal. For positive integer D, set $R_0 = 2^{-j}$, where $j = \lfloor \log_2 D \rfloor + 1$ is its bit length. Since $2^{j-1} \le D < 2^j$, the residual satisfies $0 < e_0 \le 1/2$ and shrinks under real squaring. Find j by counting leading zeros; form R_0 by exponent placement, or encode it as 2^{q-j} with $q \ge j$ fractional bits (one shift). For $D = 1101_2 = 13$, $j = 4$ gives $R_0 = 1/16$ and $e_0 = 3/16$. The 2-adic seed $R_0 = 13$ gives $e_0 = -168 = -21 \cdot 2^3$, whose ordinary magnitude grows. Keep leading bits using scaled wide products and guard information; an exact remainder check can certify final FP rounding. Finite arithmetic adds rounding or truncation errors to the exact identity.

Bit reversal does not convert one inverse into the other. For $m = 7/4$, $7^{-1} \bmod 256 = 183 = 10110111_2$. Reversal gives $11101101_2 = 237$, whereas the eight-fractional-bit real reciprocal is $\lfloor 256(4/7) \rfloor = 146$. Recovering a rational from modular data requires sufficient precision and certified reconstruction bounds.

The shared precision law can be stated with decreasing error sets:

$$F_r F_s \subseteq F_{r+s}, \qquad \begin{cases} F_r = 2^r \mathbb{Z}_2 & \text{(2-adic)}, \\ F_r = t^r \mathbb{F}_2[[t]] & \text{(power series)}, \\ F_r = \{x : |x| \le 2^{-r}\} & \text{(real)}. \end{cases}$$

If $e_0 \in F_r$ with $r > 0$, exact refinement gives $e_k \in F_{2^k r}$. For nonzero errors, 2-adic valuation and real $-\log_2 |e_k|$ double at each step. The first two families are ideals; the real family supplies multiplicative size bounds, not additive subgroups. A shared convergence identity therefore coexists with different notions of known digits.

Implemented boundary. The ARM64 demo now has a separate integer-only FP64 path for add, subtract, multiply and divide, with four rounding modes, gradual underflow, special values and software flags. Multiplication uses the full `MUL`/`UMULH` product; division currently uses integer quotient/remainder steps, not reciprocal iteration. The original p-adic path keeps its modular semantics. Native and exact-rational checks validate the tested cases; disassembly confirms no FP instructions in the new core. See the implementation and reproducible checks. Software FP arithmetic is established practice [3]; this demonstrates the shared integer machinery, without claiming new arithmetic, formal completeness, or measured hardware savings.

Architectural conclusion. Keep the integer multiplier. Choose the operand encoding, retained product region, normalization and rounding policy for each arithmetic. Carry direction is an indexing question; precision is a semantic one.

Selected references

1. Arm, ARMv8-A A64 ISA Overview: integer `MUL` and `UMULH`.
2. FLINT, Hensel lifting of inverses: modular reciprocal refinement.
3. J. Hauser, Berkeley SoftFloat: software binary FP, rounding and exception policies.
4. D. J. Bernstein, Fast multiplication and its applications, MSRI Publications 44 (2008), §§5–7, 17.